```
clc; clear; close all;
calib_folder='calib';
eval_folder='eval';
trial_length=[0.5 5];
amp_thresh_sd=6;
fbands=[8  12;
        10 14;
        12 16;
       % 18 22;
        20 24;
       % 22 26;
        24 30];
target_channels={'C3','C4','Cz'};
n_csp_per_band=2;
calib_files=dir(fullfile(calib_folder, '*.mat'));
file_accuracies=nan(length(calib_files), 1);
file_kappas=nan(length(calib_files), 1);
disp('========== PHASE 1: Cross-Validation (per file) ==========');
```

```
========== PHASE 1: Cross-Validation (per file) ==========
```

```
for f=1:length(calib_files)
    load(fullfile(calib_files(f).folder, calib_files(f).name));
    cnt=0.1 * double(cnt);
    fs=nfo.fs;
    idx=find(ismember(nfo.clab, target_channels));
    cnt=cnt(:, idx);
    n_ch=size(cnt, 2);
    if n_ch==0
        warning('File %s: no target channels found, skipping.',
calib_files(f).name);
        continue;
    end
    global_std=std(cnt(:));
    amp_thresh=amp_thresh_sd * global_std;
    fprintf('\n--- FILE: %s ---\n', calib_files(f).name);
    fprintf('Global std: %.2f µV | Threshold: %.2f µV\n', global_std, amp_thresh);
    if isfield(mrk, 'y') && ~isempty(mrk.y)
        raw_labels=mrk.y(:);
    elseif isfield(mrk, 'event') && isfield(mrk.event, 'desc')
        raw_labels=mrk.event.desc(:);
    elseif isfield(mrk, 'desc')
        raw_labels=mrk.desc(:);
    else
        warning('File %s: no label field found, skipping.', calib_files(f).name);
        continue;
    end
    samples=round(trial_length * fs);
    num_trials=length(mrk.pos);
    X_trials={};
```

```
    Y_all=[];
    rejected_count=0;
    for j=1:num_trials
        start_idx=mrk.pos(j) + samples(1);
        end_idx=mrk.pos(j) + samples(2);
        if end_idx>size(cnt, 1), continue; end
        epoch=cnt(start_idx:end_idx, :)';
        if max(abs(epoch(:)))>amp_thresh
            rejected_count=rejected_count + 1;
            continue;
        end
        X_trials{end+1}=epoch;
        Y_all(end+1, 1)=raw_labels(j);
    end
    fprintf('Trials: %d total | %d rejected | %d kept\n', ...
            num_trials, rejected_count, length(Y_all));
    u=unique(Y_all);
    if length(u)<2
        warning('File %s: fewer than 2 classes, skipping.', calib_files(f).name);
        continue;
    end
    counts=arrayfun(@(x) sum(Y_all==x), u);
    [~, ord]=sort(counts, 'descend');
    class1=u(ord(1));
    class2=u(ord(2));
    keep=(Y_all==class1) | (Y_all==class2);
    X_trials=X_trials(keep);
    Y_all=Y_all(keep);
    N=length(Y_all);
    min_per_class=min(arrayfun(@(x) sum(Y_all==x), [class1 class2]));
    K=min(5, min_per_class);
    if K<2
        warning('File %s: too few trials (%d), skipping.', calib_files(f).name,
min_per_class);
        continue;
    end
    %% FBCSP features
    n_csp=min(n_csp_per_band, floor(n_ch / 2));
    features_all=zeros(N, 2 * n_csp * size(fbands, 1));
    for b=1:size(fbands, 1)
        band=fbands(b, :);
        [bb, aa]=butter(4, band / (fs/2), 'bandpass');
        X_filt=cellfun(@(x) filtfilt(bb, aa, x')', X_trials, 'UniformOutput',
false);
        c1_idx=find(Y_all==class1);
        c2_idx=find(Y_all==class2);
        C1=zeros(n_ch);
        for i=1:length(c1_idx)
            X=X_filt{c1_idx(i)};
            C1=C1 + (X * X') / trace(X * X');
```

```
        end
        C1=C1 / length(c1_idx);
        C2=zeros(n_ch);
        for i=1:length(c2_idx)
            X=X_filt{c2_idx(i)};
            C2=C2+(X*X')/trace(X * X');
        end
        C2=C2 / length(c2_idx);

        [EVec, EVal]=eig(C1, C1 + C2);
        [~, ind]=sort(diag(EVal), 'descend');
        W_b=EVec(:, ind);
        W_b=[W_b(:, 1:n_csp), W_b(:, end-n_csp+1:end)];

        feat_col_start=(b-1) * 2*n_csp + 1;
        for i=1:N
            Z=W_b' * X_filt{i};
            varZ=var(Z, 0, 2);
            features_all(i, feat_col_start : feat_col_start + 2*n_csp - 1)= ...
                log(varZ / sum(varZ));
        end
    end

    Y_cat=categorical(Y_all);
    cv=cvpartition(N, 'KFold', K);

    acc=zeros(K, 1);
    kappa_vals=zeros(K, 1);

    for i=1:K
        trainIdx=training(cv, i);
        testIdx=test(cv, i);

        if length(unique(Y_all(testIdx)))<2
            acc(i)=NaN; kappa_vals(i)=NaN; continue;
        end

        X_tr=features_all(trainIdx, :);
        X_te=features_all(testIdx, :);

        mu_tr=mean(X_tr);
        sigma_tr=std(X_tr);
        sigma_tr(sigma_tr==0)=1;

        X_tr=(X_tr - mu_tr) ./ sigma_tr;
        X_te=(X_te - mu_tr) ./ sigma_tr;

        model_cv=fitcdiscr(X_tr, Y_cat(trainIdx), 'DiscrimType', 'linear');
        pred=predict(model_cv, X_te);
```

```
        acc(i)=mean(pred==Y_cat(testIdx));

        true_lab=Y_cat(testIdx);
        classes=categories(Y_cat);
        C_mat=confusionmat(true_lab, pred, 'Order', classes);
        n_total=sum(C_mat(:));
        p_o=sum(diag(C_mat)) / n_total;
        p_e=sum(sum(C_mat, 1) .* sum(C_mat, 2)') / n_total^2;
        kappa_vals(i)=(p_o - p_e) / (1 - p_e);
    end

    file_accuracies(f)=mean(acc(~isnan(acc))) * 100;
    file_kappas(f)=mean(kappa_vals(~isnan(kappa_vals)));

    fprintf('CV Result -> Acc: %.1f%% | Kappa: %.3f\n', ...
            file_accuracies(f), file_kappas(f));
end
```

```
--- FILE: BCICIV_calib_ds1a.mat ---
Global std: 171.75 µV | Threshold: 1030.51 µV
Trials: 200 total | 0 rejected | 200 kept
CV Result -> Acc: 82.5% | Kappa: 0.651
--- FILE: BCICIV_calib_ds1b.mat ---
Global std: 46.25 µV | Threshold: 277.52 µV
Trials: 200 total | 2 rejected | 198 kept
CV Result -> Acc: 73.8% | Kappa: 0.476
--- FILE: BCICIV_calib_ds1f.mat ---
Global std: 70.64 µV | Threshold: 423.87 µV
Trials: 200 total | 0 rejected | 200 kept
CV Result -> Acc: 72.5% | Kappa: 0.447
--- FILE: BCICIV_calib_ds1g.mat ---
Global std: 70.32 µV | Threshold: 421.89 µV
Trials: 200 total | 2 rejected | 198 kept
CV Result -> Acc: 89.9% | Kappa: 0.796
```

```
%% ===== CV Summary =====
valid=~isnan(file_accuracies);
fprintf('\n============ CV SUMMARY ============\n');
```

```
============ CV SUMMARY ============
```

```
for f=1:length(calib_files)
    if valid(f)
        fprintf('  %-30s | Acc: %5.1f%% | Kappa: %.3f\n', ...
                calib_files(f).name, file_accuracies(f), file_kappas(f));
    else
        fprintf('  %-30s | SKIPPED\n', calib_files(f).name);
    end
end
```

```
  BCICIV_calib_ds1a.mat          | Acc:  82.5% | Kappa: 0.651
  BCICIV_calib_ds1b.mat          | Acc:  73.8% | Kappa: 0.476
  BCICIV_calib_ds1f.mat          | Acc:  72.5% | Kappa: 0.447
  BCICIV_calib_ds1g.mat          | Acc:  89.9% | Kappa: 0.796
```

```
fprintf('\nMean Accuracy : %.1f%% +/- %.1f%%\n', ...
```

```
        mean(file_accuracies(valid)), std(file_accuracies(valid)));
```

```
Mean Accuracy : 79.7% +/- 8.1%
```

```
fprintf('Mean Kappa    : %.3f +/- %.3f\n', ...
        mean(file_kappas(valid)), std(file_kappas(valid)));
```

```
Mean Kappa    : 0.592 +/- 0.163
```

```
%% ===== PHASE 2: Train Final Model on ALL Calibration Data Combined =====
fprintf('\n========== PHASE 2: Training Final Model on All Calib Data
==========\n');
```

```
========== PHASE 2: Training Final Model on All Calib Data ==========
```

```
X_all_combined=[];
Y_all_combined=[];
all_epochs={};
all_labels=[];
fs_global=[];
n_ch_global=[];
for f=1:length(calib_files)
    load(fullfile(calib_files(f).folder, calib_files(f).name));
    cnt=0.1 * double(cnt);
    fs=nfo.fs;
    if isempty(fs_global)
        fs_global=fs;
    elseif fs~=fs_global
        warning('File %s: fs mismatch (%d vs %d), skipping in final model.', ...
                calib_files(f).name, fs, fs_global);
        continue;
    end
    idx=find(ismember(nfo.clab, target_channels));
    cnt=cnt(:, idx);
    n_ch=size(cnt, 2);
    if n_ch==0, continue; end
    if isempty(n_ch_global)
        n_ch_global=n_ch;
    elseif n_ch~=n_ch_global
        warning('File %s: channel count mismatch, skipping.', calib_files(f).name);
        continue;
    end
    global_std=std(cnt(:));
    amp_thresh=amp_thresh_sd * global_std;
    if isfield(mrk, 'y') && ~isempty(mrk.y)
        raw_labels=mrk.y(:);
    elseif isfield(mrk, 'event') && isfield(mrk.event, 'desc')
        raw_labels=mrk.event.desc(:);
    elseif isfield(mrk, 'desc')
        raw_labels=mrk.desc(:);
    else
        continue;
```

```
    end
    samples=round(trial_length * fs);
    num_trials=length(mrk.pos);
    for j=1:num_trials
        start_idx=mrk.pos(j) + samples(1);
        end_idx=mrk.pos(j) + samples(2);
        if end_idx>size(cnt, 1), continue; end
        epoch=cnt(start_idx:end_idx, :)';
        if max(abs(epoch(:)))>amp_thresh, continue; end
        all_epochs{end+1}=epoch;
        all_labels(end+1, 1)=raw_labels(j);
    end
end
fprintf('Total epochs collected across all calib files: %d\n', length(all_labels));
```

```
Total epochs collected across all calib files: 796
```

```
u_global=unique(all_labels);
counts_g=arrayfun(@(x) sum(all_labels==x), u_global);
[~, ord]=sort(counts_g, 'descend');
class1_g=u_global(ord(1));
class2_g=u_global(ord(2));
fprintf('Global classes: %d (%d trials) vs %d (%d trials)\n', ...
        class1_g, counts_g(ord(1)), class2_g, counts_g(ord(2)));
```

```
Global classes: -1 (398 trials) vs 1 (398 trials)
```

```
keep_g=(all_labels==class1_g) | (all_labels==class2_g);
all_epochs=all_epochs(keep_g);
all_labels=all_labels(keep_g);
N_total=length(all_labels);
%% FBCSP
n_csp=min(n_csp_per_band, floor(n_ch_global / 2));
n_feats=2 * n_csp * size(fbands, 1);
X_all=zeros(N_total, n_feats);
W_all=cell(size(fbands, 1), 1);
c1_idx_g=find(all_labels==class1_g);
c2_idx_g=find(all_labels==class2_g);
for b=1:size(fbands, 1)
    band=fbands(b, :);
    [bb, aa]=butter(4, band / (fs_global/2), 'bandpass');
    X_filt=cellfun(@(x) filtfilt(bb, aa, x')', all_epochs, 'UniformOutput', false);
    C1=zeros(n_ch_global);
    for i=1:length(c1_idx_g)
        Xc=X_filt{c1_idx_g(i)};
        C1=C1 + (Xc * Xc') / trace(Xc * Xc');
    end
    C1=C1/length(c1_idx_g);
    C2=zeros(n_ch_global);
    for i=1:length(c2_idx_g)
        Xc=X_filt{c2_idx_g(i)};
```

```
        C2=C2 + (Xc * Xc') / trace(Xc * Xc');
    end
    C2=C2/length(c2_idx_g);
    [EVec, EVal]=eig(C1, C1 + C2);
    [~, ind]=sort(diag(EVal), 'descend');
    W_b=EVec(:, ind);
    W_b=[W_b(:, 1:n_csp), W_b(:, end-n_csp+1:end)];
    W_all{b}=W_b;   % save for evaluation
    feat_col_start=(b-1) * 2*n_csp + 1;
    for i=1:N_total
        Z=W_b'*X_filt{i};
        varZ=var(Z, 0, 2);
        X_all(i, feat_col_start : feat_col_start + 2*n_csp - 1)= ...
            log(varZ / sum(varZ));
    end
end
mu_X=mean(X_all);
sigma_X=std(X_all);
sigma_X(sigma_X==0)=1;
X_all_norm=(X_all - mu_X) ./ sigma_X;
Y_all_labels=all_labels;
fprintf('Training LDA on %d trials...\n', N_total);
```

```
Training LDA on 796 trials...
```

```
% final_model = fitcsvm(X_all_norm, Y_all_labels, ...
    % 'KernelFunction', 'rbf', ...
    % 'Standardize',    false, ...   % already normalized
    % 'OptimizeHyperparameters', 'none');
final_model=fitcdiscr(X_all_norm, Y_all_labels, ...
    'DiscrimType', 'pseudolinear', ...
    'Gamma', 0.9);    % 0=full LDA, 1=diagonal; tune between 0.1-0.9
fprintf('Final model trained. Classes: %d vs %d\n', class1_g, class2_g);
```

```
Final model trained. Classes: -1 vs 1
```

```
%% ===== PHASE 3: Predict on Evaluation Files =====
fprintf('\n========== PHASE 3: Evaluation ==========\n');
```

```
========== PHASE 3: Evaluation ==========
```

```
eval_files=dir(fullfile(eval_folder, '*.mat'));
if isempty(eval_files)
    warning('No evaluation files found in folder: %s', eval_folder);
end
for i=1:length(eval_files)
    fprintf('\n--- Eval File: %s ---\n', eval_files(i).name);
    load(fullfile(eval_files(i).folder, eval_files(i).name));
    cnt=0.1 * double(cnt);
    fs=nfo.fs;
    idx=find(ismember(nfo.clab, target_channels));
    cnt=cnt(:, idx);
```

```
    if size(cnt, 2)~=n_ch_global
        warning('Eval file %s: channel count mismatch (%d vs %d expected),
skipping.', ...
                eval_files(i).name, size(cnt, 2), n_ch_global);
        continue;
    end
    samples=round(trial_length * fs);
    epoch_len=samples(2);
    step_size=epoch_len;
    n_samples=size(cnt, 1);
    window_starts=1 : step_size : (n_samples - epoch_len + 1);
    n_windows=length(window_starts);
    if n_windows==0
        warning('Eval file %s: signal too short for even one window, skipping.', ...
                eval_files(i).name);
        continue;
    end
    features_eval=zeros(n_windows, n_feats);
    for b=1:size(fbands, 1)
        band=fbands(b, :);
        [bb, aa]=butter(4, band / (fs/2), 'bandpass');
        cnt_filt=filtfilt(bb, aa, cnt);
        feat_col_start=(b-1) * 2*n_csp + 1;
        W_b=W_all{b};
        for j=1:n_windows
            s_idx=window_starts(j);
            e_idx=s_idx + epoch_len - 1;
            epoch=cnt_filt(s_idx:e_idx, :)';
            Z=W_b'*epoch;
            varZ=var(Z, 0, 2);
            features_eval(j, feat_col_start : feat_col_start + 2*n_csp - 1)= ...
                log(varZ / sum(varZ));
        end
    end
    features_eval_norm=(features_eval - mu_X) ./ sigma_X;
    predictions=predict(final_model, features_eval_norm);
    fprintf('Window predictions (%d windows):\n', n_windows);
    disp(predictions');
    pred_mode=mode(predictions);
    fprintf('Majority vote prediction for file: %d\n', pred_mode);
    class_counts=arrayfun(@(c) sum(predictions==c), [class1_g, class2_g]);
    fprintf('  Class %d: %d windows (%.1f%%)\n', class1_g, class_counts(1), ...
            100*class_counts(1)/n_windows);
    fprintf('  Class %d: %d windows (%.1f%%)\n', class2_g, class_counts(2), ...
            100*class_counts(2)/n_windows);
end
```

```
--- Eval File: BCICIV_eval_ds1a.mat ---
Window predictions (481 windows):
     1    -1    -1     1    -1     1    -1    -1     1     1     1    -1     1     1    -1     1     1     1     1
Majority vote prediction for file: 1
```

```
  Class -1: 80 windows (16.6%)
  Class 1: 401 windows (83.4%)
--- Eval File: BCICIV_eval_ds1b.mat ---
Window predictions (493 windows):
     1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1
Majority vote prediction for file: -1
  Class -1: 415 windows (84.2%)
  Class 1: 78 windows (15.8%)
--- Eval File: BCICIV_eval_ds1c.mat ---
Window predictions (466 windows):
     1     1     1     1     1     1     1     1     1     1     1     1    -1     1     1     1     1     1     1
Majority vote prediction for file: 1
  Class -1: 46 windows (9.9%)
  Class 1: 420 windows (90.1%)
--- Eval File: BCICIV_eval_ds1d.mat ---
Window predictions (490 windows):
    -1    -1     1    -1    -1    -1    -1    -1    -1    -1    -1    -1     1     1    -1    -1    -1     1     1
Majority vote prediction for file: -1
  Class -1: 313 windows (63.9%)
  Class 1: 177 windows (36.1%)
--- Eval File: BCICIV_eval_ds1e.mat ---
Window predictions (476 windows):
    -1     1    -1     1     1    -1    -1    -1    -1    -1    -1    -1    -1     1    -1    -1    -1     1     1
Majority vote prediction for file: -1
  Class -1: 352 windows (73.9%)
  Class 1: 124 windows (26.1%)
--- Eval File: BCICIV_eval_ds1f.mat ---
Window predictions (478 windows):
     1    -1    -1    -1     1    -1    -1     1    -1    -1     1     1     1    -1     1    -1    -1    -1    -1
Majority vote prediction for file: -1
  Class -1: 260 windows (54.4%)
  Class 1: 218 windows (45.6%)
--- Eval File: BCICIV_eval_ds1g.mat ---
Window predictions (487 windows):
    -1    -1     1     1     1    -1     1     1    -1    -1    -1    -1    -1    -1    -1    -1    -1    -1     1
Majority vote prediction for file: -1
  Class -1: 268 windows (55.0%)
  Class 1: 219 windows (45.0%)
```

```
%% ===== PHASE 4: Test on Labeled Test Data (Robustness Check) =====
fprintf('\n========== PHASE 4: Labeled Test Data Robustness Check ==========\n');
```

```
========== PHASE 4: Labeled Test Data Robustness Check ==========
```

```
test_folder='test';
test_files=dir(fullfile(test_folder, '*.mat'));
if isempty(test_files)
    warning('No test files found in folder: %s', test_folder);
end
all_test_true=[];
all_test_pred=[];
for i=1:length(test_files)
    fprintf('\n--- Test File: %s ---\n', test_files(i).name);
    load(fullfile(test_files(i).folder, test_files(i).name));
    cnt=0.1 * double(cnt);
    fs=nfo.fs;
    idx=find(ismember(nfo.clab, target_channels));
    cnt=cnt(:, idx);
    if size(cnt, 2)~=n_ch_global
```

```
        warning('Test file %s: channel count mismatch (%d vs %d expected),
skipping.', ...
                test_files(i).name, size(cnt, 2), n_ch_global);
        continue;
    end
    if isfield(mrk, 'y') && ~isempty(mrk.y)
        raw_labels=mrk.y(:);
    elseif isfield(mrk, 'event') && isfield(mrk.event, 'desc')
        raw_labels=mrk.event.desc(:);
    elseif isfield(mrk, 'desc')
        raw_labels=mrk.desc(:);
    else
        warning('Test file %s: no label field found, skipping.',
test_files(i).name);
        continue;
    end
    global_std=std(cnt(:));
    amp_thresh=amp_thresh_sd * global_std;
    samples=round(trial_length * fs);
    num_trials=length(mrk.pos);
    X_test_epochs={};
    Y_test=[];
    rejected_count=0;
    for j=1:num_trials
        start_idx=mrk.pos(j) + samples(1);
        end_idx=mrk.pos(j) + samples(2);
        if end_idx>size(cnt, 1), continue; end
        epoch=cnt(start_idx:end_idx, :)';
        if max(abs(epoch(:)))>amp_thresh
            rejected_count=rejected_count + 1;
            continue;
        end
        X_test_epochs{end+1}=epoch;
        Y_test(end+1, 1)=raw_labels(j);
    end
    fprintf('Trials: %d total | %d rejected | %d kept\n', ...
            num_trials, rejected_count, length(Y_test));
    if isempty(Y_test)
        warning('Test file %s: no valid trials after artifact rejection,
skipping.', ...
                test_files(i).name);
        continue;
    end
    keep_test=(Y_test==class1_g) | (Y_test==class2_g);
    X_test_epochs=X_test_epochs(keep_test);
    Y_test=Y_test(keep_test);
    N_test=length(Y_test);
    if N_test==0
        warning('Test file %s: no trials matching training classes (%d, %d),
skipping.', ...
```

```
                test_files(i).name, class1_g, class2_g);
        continue;
    end
    fprintf('Trials matching training classes: %d\n', N_test);
    features_test=zeros(N_test, n_feats);
    for b=1:size(fbands, 1)
        band=fbands(b, :);
        [bb, aa]=butter(4, band / (fs/2), 'bandpass');
        X_filt=cellfun(@(x) filtfilt(bb, aa, x')', ...
                       X_test_epochs, 'UniformOutput', false);

        W_b=W_all{b};
        feat_col_start=(b-1) * 2*n_csp + 1;
        for j=1:N_test
            Z=W_b' * X_filt{j};
            varZ=var(Z, 0, 2);
            features_test(j, feat_col_start : feat_col_start + 2*n_csp - 1)= ...
                log(varZ / sum(varZ));
        end
    end
    features_test_norm=(features_test - mu_X) ./ sigma_X;
    predictions_test=predict(final_model, features_test_norm);
    acc_test=mean(predictions_test==Y_test) * 100;
    classes=[class1_g; class2_g];
    C_mat=confusionmat(Y_test, predictions_test, 'Order', classes);
    n_total=sum(C_mat(:));
    p_o=sum(diag(C_mat)) / n_total;
    p_e=sum(sum(C_mat, 1) .* sum(C_mat, 2)') / n_total^2;
    kappa=(p_o - p_e) / (1 - p_e);
    fprintf('Accuracy : %.1f%%\n', acc_test);
    fprintf('Kappa    : %.3f\n',   kappa);
    fprintf('Confusion Matrix (rows=true, cols=pred) | Classes: %d, %d\n', ...
            class1_g, class2_g);
    disp(C_mat);
    for c=1:length(classes)
        cls=classes(c);
        cls_mask=(Y_test==cls);
        cls_acc=mean(predictions_test(cls_mask)==Y_test(cls_mask)) * 100;
        fprintf('  Class %d -> Accuracy: %.1f%% (%d/%d trials correct)\n', ...
                cls, cls_acc, sum(predictions_test(cls_mask)==cls), sum(cls_mask));
    end
    all_test_true=[all_test_true; Y_test];
    all_test_pred=[all_test_pred; predictions_test];
end
```

```
--- Test File: BCICIV_calib_ds1c.mat ---
Trials: 200 total | 0 rejected | 200 kept
Trials matching training classes: 200
Accuracy : 50.0%
Kappa    : 0.000
Confusion Matrix (rows=true, cols=pred) | Classes: -1, 1
```

```
   100     0
   100     0
  Class -1 -> Accuracy: 100.0% (100/100 trials correct)
  Class 1 -> Accuracy: 0.0% (0/100 trials correct)
--- Test File: BCICIV_calib_ds1d.mat ---
Trials: 200 total | 0 rejected | 200 kept
Trials matching training classes: 200
Accuracy : 72.0%
Kappa    : 0.440
Confusion Matrix (rows=true, cols=pred) | Classes: -1, 1
    51    49
     7    93
  Class -1 -> Accuracy: 51.0% (51/100 trials correct)
  Class 1 -> Accuracy: 93.0% (93/100 trials correct)
--- Test File: BCICIV_calib_ds1e.mat ---
Trials: 200 total | 0 rejected | 200 kept
Trials matching training classes: 200
Accuracy : 89.5%
Kappa    : 0.790
Confusion Matrix (rows=true, cols=pred) | Classes: -1, 1
   100     0
    21    79
  Class -1 -> Accuracy: 100.0% (100/100 trials correct)
  Class 1 -> Accuracy: 79.0% (79/100 trials correct)
```

```
fprintf('\n============ GLOBAL TEST SUMMARY ============\n');
```

```
============ GLOBAL TEST SUMMARY ============
```

```
if isempty(all_test_true)
    fprintf('No valid test trials found across all test files.\n');
else
    overall_acc=mean(all_test_pred==all_test_true) * 100;

    classes=[class1_g; class2_g];
    C_global=confusionmat(all_test_true, all_test_pred, 'Order', classes);
    n_total=sum(C_global(:));
    p_o=sum(diag(C_global)) / n_total;
    p_e=sum(sum(C_global, 1) .* sum(C_global, 2)') / n_total^2;
    kappa_global=(p_o - p_e) / (1 - p_e);

    fprintf('Total test trials : %d\n',   n_total);
    fprintf('Overall Accuracy  : %.1f%%\n', overall_acc);
    fprintf('Overall Kappa     : %.3f\n',   kappa_global);
    fprintf('\nGlobal Confusion Matrix (rows=true, cols=pred) | Classes: %d, %d\n',
...
            class1_g, class2_g);
    disp(C_global);


    %% Comparison: CV vs Final Test
    fprintf('\n--- Performance Comparison ---\n');
    fprintf('  CV Mean Accuracy  (calib) : %.1f%% +/- %.1f%%\n', ...
            mean(file_accuracies(valid)), std(file_accuracies(valid)));
    fprintf('  CV Mean Kappa     (calib) : %.3f +/- %.3f\n', ...
```

```
            mean(file_kappas(valid)), std(file_kappas(valid)));
    fprintf('  Final Test Accuracy       : %.1f%%\n', overall_acc);
    fprintf('  Final Test Kappa          : %.3f\n',   kappa_global);

    gap=mean(file_accuracies(valid)) - overall_acc;
    fprintf('Generalization gap: %.1f%%\n', gap)
    % if abs(gap)<=5
    %     fprintf('  Generalization gap: %.1f%% -> Good generalization.\n', gap);
    % elseif gap>5
    %     fprintf('  Generalization gap: %.1f%% -> Possible overfitting.\n', gap);
    % else
    %     fprintf('  Generalization gap: %.1f%% -> Test data may differ from calib
distribution.\n', gap);
    % end
end
```

```
Total test trials : 600
Overall Accuracy  : 70.5%
Overall Kappa     : 0.410
Global Confusion Matrix (rows=true, cols=pred) | Classes: -1, 1
   251    49
   128   172
--- Performance Comparison ---
  CV Mean Accuracy  (calib) : 79.7% +/- 8.1%
  CV Mean Kappa     (calib) : 0.592 +/- 0.163
  Final Test Accuracy       : 70.5%
  Final Test Kappa          : 0.410
Generalization gap: 9.2%
```

```
fprintf('\n========== Done ==========\n');
```

```
========== Done ==========
```